

SEW-Offload: 面向昇腾 NPU 的 MoE Expert Offloading

Static Expert-Slot Windowing for Ascend MoE Offloading

不要把 NPU 当 GPU 用： 重点研究如何利用 Ascend 的差异化特性重塑 MoE expert offloading 的设计空间，并据此重新设计专家权重驻留、搬运、窗口化执行与预取机制。

What is your problem

问题场景：在 Ascend NPU 上部署大规模 MoE，expert 权重无法全量常驻 HBM

MoE 每个 token 只激活少量 expert，但完整 expert 参数集仍然需要被存放和访问。在 Ascend 单卡或低 HBM 预算下，dense 权重、KV cache 和中间 buffer 已占用大量 HBM，剩余空间只能容纳部分 expert。于是，推理过程从“所有 expert 都在 NPU 上等待执行”变成“router 本轮请求的 expert 可能需要从 host 侧取回”。

现有方法的 GPU 假设

- **expert caching**: 把 HBM 看作 expert cache，热 expert 常驻，冷 expert 放在 host；
- **host-to-device prefetching**: 根据历史路由或预测提前把可能需要的 expert 搬到设备；
- **CUDA stream overlap**: 依赖动态 kernel、stream 并发和异步拷贝隐藏传输开销。

为什么在 Ascend 上不完整

- **稳定 tensor 地址**: expert 动态装载会改变权重入口，削弱地址绑定和 buffer 复用；
- **固定 shape 与图重放**: active expert 和 token count 每轮变化，放大 graph/static-kernel 变体；
- **显式调度的数据移动**: H2D/MTE 搬运若不能被明确编排，会暴露为计算等待；
- **结果**: HBM 压力被缓解，但 NPU 静态复用能力下降，吞吐下降、尾延迟抖动。

核心问题定义: 现有 MoE 卸载通常把 expert 权重视为动态缓存对象，通过替换、预取和流级拷贝/计算重叠降低传输开销；但在 HBM 受限的 Ascend NPU 上，动态加载到任意 device buffer 会与**稳定权重地址、固定执行窗口、图重放和显式数据移动调度**冲突。结果是：要么 host-to-HBM expert 加载暴露在关键路径上，要么放弃高效 NPU 执行所需的静态规整性。

Why it matters?

MoE、Ascend 与 offloading 三个层面的必要性

随着 DeepSeek-V3/R1、Qwen-MoE、Mixtral 等模型普及，MoE 已成为大模型扩展的重要方向。问题不再只是能不能训练更大的模型，而是如何让超大 expert 参数在真实硬件上高效服务。

1. 为什么 MoE 很重要

- **扩展参数规模：**拥有巨专家参数池，但每个 token 只激活少量 expert；
- **提高推理性价比：**相比同等总参数 dense 模型，用稀疏激活降低单 token 计算量；
- **代表前沿趋势：**DeepSeek、Qwen、Mixtral 表明，大规模 MoE 已成为高能力模型的重要形态。

2. 为什么 Ascend 上重要

- **国产算力主战场：**生产环境依赖 Ascend 910B/910C，MoE 能否跑好直接影响算力可用性；
- **不能只复制 GPU：**Ascend 更强调稳定地址、固定 shape、图重放和显式搬运调度；
- **需要 NPU-native 路线：**把 MoE 适配 Ascend，才能释放 AIC/AIV/MTE 等硬件能力。

3. 为什么 offload 重要

- **HBM 是硬约束：**expert 总权重远超单卡可用 HBM，dense 权重和 KV cache 继续挤占空间；
- **部署取决于驻留策略：**没有 offload，大 MoE 只能依赖更多卡或更小模型，应用门槛显著提高；
- **Ascend offload 更难：**动态加载 expert 会破坏地址稳定与图复用，必须重新设计 slot、窗口和预取流水。

核心动机：MoE 决定了前沿模型的扩展方向，Ascend 决定了这些模型在国产算力上的落地能力；而 expert offloading 决定了在 HBM 受限时，能否把“参数很大但激活稀疏”的 MoE 真正部署起来。SEW-Offload 的价值正是在 Ascend 约束下，把动态 expert 工作集转化为可复用、可预取、可流水化的固定 slot 执行窗口。

Why existing work fails?

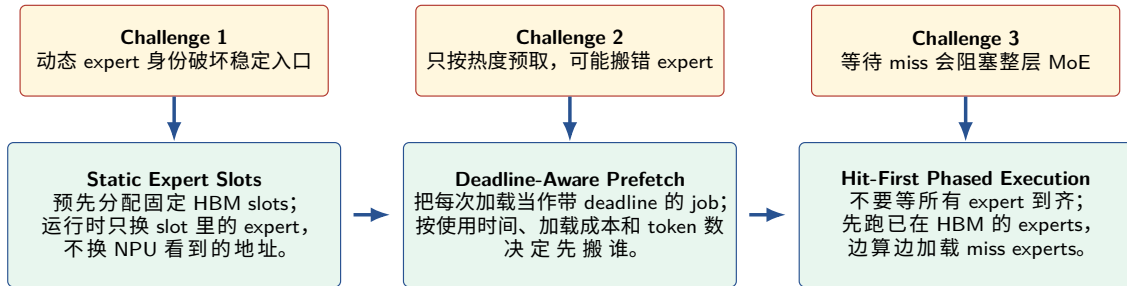
现有工作各自解决了局部问题，但没有同时覆盖 “Ascend + HBM 受限 + expert offload”。

工作类型	现有方法怎么做	在我们场景下缺什么	导致的问题
GPU MoE offload	把 HBM 当 expert cache；基于历史/预测做 prefetch；用 CUDA stream 和异步拷贝隐藏 H2D。	默认动态 kernel、动态权重对象和 stream overlap 可接受；缺少 Ascend 稳定地址、固定 shape、图重放与 MTE 编排。	HBM 压力转化为动态入口和搬运 stall，graph/static kernel 复用收益下降。
Ascend 全量 MoE	vLLM Ascend 已支持 routing 到 per-expert count/grouped execution，并用 npu_grouped_matmul 执行。	假设 expert 权重全量在 HBM；缺少 host store、resident slot、miss/prefetch/replacement 和 phased execution。	后端能高效执行 grouped MoE，但无法处理“缺失 expert 如何进入 NPU”。
非 Ascend NPU MoE	如 NPUMoE 面向 Apple ANE：把 dense/static compute offload 到 NPU，动态算子走 CPU/GPU fallback，并用 static tiers/grouped execution/graph residency。	目标硬件、内存体系和 runtime 不同；重点是 ANE 图执行与 fallback，不是 Ascend host-to-HBM expert offload。	有静态化启发，但不能直接解决 Ascend 上 expert 权重动态装载和稳定入口问题。

研究空白： 缺少面向 Ascend 的 **HBM 受限 expert offloading**：不改变 router/top-k/token 语义，同时处理权重驻留、搬运隐藏和稳定执行入口。

What is your key idea?

把 expert offloading 从 “cache hit rate 问题” 改写为 “可静态化、可预取、可分阶段执行的调度问题”。



核心转换： 固定 slot 提供稳定入口；deadline-aware prefetch 解决 “先搬谁”；hit-first phased execution 解决 “miss 已发生时怎么不整层等待”。

Old assumption fails: 旧 GPU 假设为什么不够

旧假设：expert cache 是主要问题

GPU MoE offloading 常把系统抽象成：

- router 产生动态 expert 集合；
- runtime 根据热度/预测决定加载哪些 expert；
- expert kernel 对当前 batch 动态执行；
- cache 命中率和传输带宽决定性能。

在 Ascend 上缺失的约束

如果只复制 GPU cache 策略，会忽略：

- graph capture 依赖稳定 tensor 形状和地址；
- static kernel 编译启动成本高，但重复执行收益大；
- MTE 搬运和 Cube 计算需要被流水化；
- 小而动态的 expert 执行会放大队列和调度开销。

重新定义 offloading： 在 Ascend 上，offloading 不只是“expert 在 host 和 device 之间移动”，还包括**权重驻留位置、slot 地址稳定性、图复用粒度、预取流水**这些 GPU 工作通常弱化的控制面。

Important correction: vLLM Ascend 已经做了什么

我们不能把已有 grouped MoE 后端当成新贡献。

路径	已有机制	含义
MC2	<code>npu_moe_distribute_dispatch</code> 返回 <code>expert_token_nums</code>	dispatch 后已有每个 expert 的累计 token 边界
AllGather	<code>npu_moe_init_routing</code> 返回 <code>expert_tokens</code>	token-level routing 被整理为 per-expert count
All2AllV	<code>torch.histc(topk_ids)</code> 统计 <code>tokens_per_expert</code>	expert 粒度统计用于通信和 GMM
MLP	<code>npu_grouped_matmul(... group_list=...)</code>	后端按 expert 分组执行矩阵乘

修正后的边界： vLLM Ascend 已经解决了“per-token assignment 到 per-expert count/grouped execution” 的表示问题。SEW-Offload 不能声称这是贡献；我们的目标是在此基础上解决HBM 不足时 expert 权重不再全量常驻的问题。

What is still missing: grouped count 不是 offloading window

已有 grouped MoE 后端

- 输入仍是每轮动态 topk_ids;
- group_list 的形状固定, 但 count 数值动态;
- GMM 根据每轮 split_value 决定 expert 的 M 维;
- 权重张量来自 layer.w13_weight/w2_weight。

Offloading 需要的新抽象

- HBM 中只有固定数量 expert_slots;
- expert_id -> slot_id 动态映射;
- slot 内权重 tensor 地址长期稳定;
- miss expert 触发加载、替换或慢速 fallback;
- slot 作为 graph/static-kernel 的稳定输入。

关键差异: grouped count 解决的是 activation 如何按 expert 执行; SEW-Offload 解决的是 expert 权重如何在 HBM 受限时以固定 slot 方式驻留、加载和复用。

Opportunity: Ascend 给 MoE offloading 的新控制面

Graph / static kernel

ACLGraph/NPUGraph 和 static kernel 更偏好固定 shape、固定 tensor 地址与有限执行变体。这意味着 expert offloading 应该尽量让 device 侧权重入口稳定。

Weight prefetch

vLLM Ascend 已经有通过额外 pipeline 预取线性层权重、降低 MTE 压力的经验。Expert slot 可以把“预取哪些权重”变成显式策略问题。

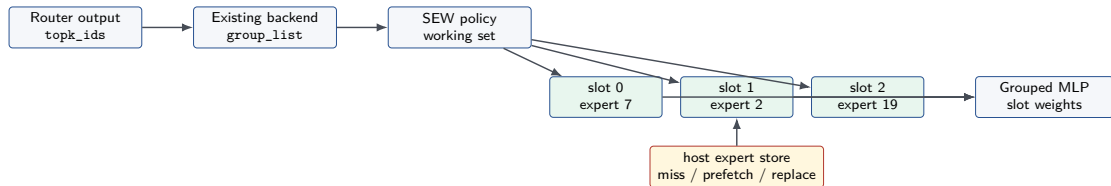
AIC/AIV/MTE 分工

MoE 执行天然包含 routing、permute、grouped GEMM、combine 与权重搬运。Ascend 的矩阵单元、向量单元和搬运单元让这些阶段有机会流水化。

Fixed GM window

如果 NPU 侧只有固定数量 slot，且每个 slot 的权重布局固定，runtime 可以把动态 expert 工作集翻译为稳定的 slot 执行窗口。

Key idea: Static Expert-Slot Windowing



核心理想： 不改变 router 选出的 expert；只是在执行前，把 active expert 映射到固定数量、固定地址的 HBM slots。命中的 expert 直接执行，未命中的 expert 触发加载/替换/降级路径。

Runtime abstraction

抽象	职责	为什么是 Ascend-specific
ExpertStore	CPU/host 侧保存完整 expert 权重, 支持按 expert 或 tile 加载	把 offloading 从一次性全量权重迁移变成可调度的数据搬运
SlotManager	维护固定数量 HBM slots 与 expert_id -> slot_id	固定 slot tensor 地址, 给 graph/static kernel 稳定入口
WindowPlanner	根据 group_list、历史热度和 HBM budget 选择 resident set	利用已有 per-expert count, 而不是重新做 routing
PrefetchEngine	在 routing/dispatch/GMM 间隙预取下一层或下一步 expert	显式利用 MTE/权重预取流水, 降低 Cube 等待
FallbackPath	处理 slot miss、超预算、预取失败和未覆盖 expert	保证不 drop token, 不改变模型语义

One iteration data flow

- ① **Routing:** 原始 router 产生 `topk_ids/topk_weights`, 不修改激活结果。
- ② **Count extraction:** 复用现有 `token_dispatcher.py` 得到 `group_list/expert_tokens`。
- ③ **Working-set planning:** 根据当前层 active experts、历史局部性和 slot budget 决定 resident set。
- ④ **Slot remap:** 将 active expert 映射到 `slot_id`, 构造 slot-local expert map。
- ⑤ **Load/prefetch:** 对 miss expert 从 host store 加载到空闲或被替换 slot。
- ⑥ **Grouped MLP:** 使用 slot 权重和现有 grouped matmul 路径执行。
- ⑦ **Combine:** 复用现有 `combine/unpermute`, 还原原始 token 顺序。

语义保证: SEW-Offload 只改变 expert 权重的驻留和访问方式, 不改变 router logits、top-k expert、gate weight 或 combine 语义。

Integration in vLLM Ascend

现有主路径

- 1 AscendFusedMoE.forward_impl
- 2 select_experts
- 3 build_fused_experts_input
- 4 moe_comm_method.fused_experts
- 5 token_dispatch -> MLP -> combine

SEW 最小插入点

- 在 fused_experts 前准备 slot weights;
- 不改 scheduler;
- 不改 model runner;
- 不重写 token dispatcher;
- 默认关闭, 通过配置开关启用。

低侵入原则: SEW-Offload 应作为 vllm_ascend/moe_offload/ 中的独立 runtime, 通过薄 hook 接入 MoE boundary; 现有 grouped count、dispatch、GMM 和 combine 尽量复用。

Policy design

MVP-0: observe

- 记录每层 active experts;
- 记录 expert_tokens;
- 分析 temporal locality;
- 不改变执行路径。

MVP-1: sync slot

- 固定 HBM slot 数;
- miss 同步加载;
- LRU/heat replacement;
- 保证 correctness。

MVP-2: prefetch

- 基于上一层/上一 token 预测;
- 对下一层 hot expert 预取;
- overlap routing/dispatch;
- 统计 stall 与命中。

MVP-3: graph-friendly window

将 slot 数、capacity tier 和权重入口固定成少量变体, 探索 ACLGraph/static kernel 对 offloaded expert path 的收益和限制。

Experiment plan

维度	设计
模型	Qwen3-30B-A3B; 优先单机单卡, 后续扩展多卡 expert parallel 对照
场景	人工限制 resident expert slots, 模拟 HBM 不足; 保留真实 KV cache 压力
Baseline 1	vLLM Ascend 原生 grouped MoE, 全量 expert 常驻
Baseline 2	简单 LRU expert cache, 无固定 slot/graph 设计
Baseline 3	GPU-style prediction/prefetch 思路移植, 不做 Ascend-specific slot 稳定化
SEW variants	sync slot、prefetch slot、capacity-tier slot、graph replay slot
Workloads	decode-heavy、mixed prefill/decode、长上下文、多请求连续批处理

Metrics and ablations

主要指标

- end-to-end throughput;
- TTFT / TPOT / p95 latency;
- HBM peak / resident expert memory;
- slot hit rate / miss penalty;
- H2D bytes and stalls;
- graph replay hit rate;
- MTE/Cube overlap。

消融实验

- slot 数量: 2/4/8/16;
- replacement: LRU、LFU、token-count weighted;
- prefetch: 无、上一层、上一 token、混合;
- capacity tier: 关闭/少量 bucket;
- graph: eager、ACLGraph、static kernel。

Expected claims

- ① **表示层 claim:** Ascend grouped MoE 已经提供 per-expert count; offloading 应复用这个信号, 而不是重新设计 routing。
- ② **系统层 claim:** 固定 expert-slot residency 能让 HBM 受限 MoE offloading 保持稳定权重入口, 减少动态权重对象破坏图复用的问题。
- ③ **硬件层 claim:** slot 化后, expert load/prefetch 可以与 routing、dispatch、GMM 形成更明确的流水, 有机会改善 MTE/Cube overlap。
- ④ **语义层 claim:** SEW-Offload 不修改 router、不减少 expert 激活、不做 drop, 因此 correctness 风险低于重路由或容量裁剪方法。

Risks and fallback

风险	表现	应对
slot miss 太多	加载阻塞吞吐，尾延迟升高	decode/prefill 分开评估；引入 token-count weighted replacement
graph 收益有限	dynamic count 仍导致图变体过多	先固定权重 slot 地址，再逐步引入 capacity tier
H2D 搬运太慢	offload 节省 HBM 但拖慢计算	量化 expert store、tile-level prefetch、异步加载
工程侵入过大	影响 vLLM Ascend 主路径稳定性	默认关闭；新包封装；先 observer，再 sync slot
正确性难验证	slot remap 引入 expert 对错风险	保留 expert-id checksum、逐层输出对齐、E2E replay 测试

Roadmap

Phase 1: Trace and evidence

实现 routed expert working-set 记录，得到 Qwen3-30B-A3B 在真实 workload 下的 expert locality、active set size、layer-to-layer overlap。

Phase 2: Minimal offloading

构建 host expert store 与固定 HBM slots，先用同步加载跑通 correctness，证明在人工 slot budget 下能替代全量 expert 常驻。

Phase 3: Ascend-specific optimization

加入 slot prefetch、capacity tier 和 graph/static-kernel 实验，验证固定 slot 地址是否能提升 replay 稳定性并降低搬运 stall。

一句话总结： SEW-Offload 的研究价值不在“让 MoE 后端按 expert 分组”，而在“当 expert 不能全量常驻 HBM 时，如何把动态 expert 工作集翻译成 Ascend 友好的固定 slot 执行窗口”。